

```
import pandas as pd
import numpy as np
```

✓ Step 1: Data Loading and Initial Exploration

Task 1: Load all datasets using Pandas

```
# load dimensions
df_customers = pd.read_csv('dataset/customers.csv')
df_location = pd.read_csv('dataset/location.csv')
df_sellers = pd.read_csv('dataset/sellers.csv')
df_products = pd.read_csv('dataset/products.csv')
df_cat_trans = pd.read_csv('dataset/category_translation.csv')

# load facts
df_orders = pd.read_csv('dataset/orders.csv')
df_order_item = pd.read_csv('dataset/order_item.csv')
df_payments = pd.read_csv('dataset/payments.csv')
df_reviews = pd.read_csv('dataset/reviews.csv')

df_customers.attrs['name'] = 'Customers'
df_location.attrs['name'] = 'Location'
df_sellers.attrs['name'] = 'Sellers'
df_products.attrs['name'] = 'Products'
df_cat_trans.attrs['name'] = 'Category Translation'
df_orders.attrs['name'] = 'Orders'
df_order_item.attrs['name'] = 'Order Items'
df_payments.attrs['name'] = 'Payments'
df_reviews.attrs['name'] = 'Reviews'
```

✓ Task 2: Inspect structure using .head(), .info(), .describe()

```
from IPython.display import display, HTML

def basic_inspect(df):
    df_name = df.attrs.get('name', '')
    display(HTML(f"<b>{' '*30}</b> Inspect Dataframe {df_name} {' '*30}</b>"))
    print('\nHead rows ----->')
    display(df.head())
    print('\nInformation ----->\n')
    display(df.info())
    print('\nDescribe -----> \n')
    display(df.describe())

basic_inspect(df_customers)
basic_inspect(df_location)
basic_inspect(df_sellers)
basic_inspect(df_products)
basic_inspect(df_cat_trans)
basic_inspect(df_orders)
basic_inspect(df_order_item)
basic_inspect(df_payments)
basic_inspect(df_reviews)
```


***** Inspect Dataframe Customers *****

Head rows ----->

	customer_id	customer_unique_id	customer_zip_code_prefix
0	06b8999e2fba1a1fbc88172c00ba8bc7	861eff4711a542e4b93843c6dd7febb0	14409
1	18955e83d337fd6b2def6b18a428ac77	290c77bc529b7ac935b93aa66c333dc3	9790
2	4e7b3e00288586ebd08712fdd0374a03	060e732b5b29e8181a18229c7b0b2b5e	1151
3	b2b6027bc5c5109e529d4dc6358b12c3	259dac757896d24d7702b9acbbff3f3c	8775
4	4f2d8ab171c80ec8364f7c12e35b23ad	345ecd01c38d18a9036ed96c73b8d066	13056

Information ----->

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
0	customer_id	99441 non-null	object
1	customer_unique_id	99441 non-null	object
2	customer_zip_code_prefix	99441 non-null	int64
3	customer_city	99441 non-null	object
4	customer_state	99441 non-null	object

dtypes: int64(1), object(4)

memory usage: 3.8+ MB

None

Describe ----->

	customer_zip_code_prefix
count	99441.000000
mean	35137.474583
std	29797.938996
min	1003.000000
25%	11347.000000
50%	24416.000000
75%	58900.000000
max	99990.000000

***** Inspect Dataframe Location *****

Head rows ----->

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng	geolocation_city	geolocation_
0	1037	-23.545621	-46.639292	sao paulo	
1	1046	-23.546081	-46.644820	sao paulo	
2	1046	-23.546129	-46.642951	sao paulo	
3	1041	-23.544392	-46.639499	sao paulo	
4	1035	-23.541578	-46.641607	sao paulo	

Information ----->

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000163 entries, 0 to 1000162

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----

```

0 geolocation_zip_code_prefix 1000163 non-null int64
1 geolocation_lat             1000163 non-null float64
2 geolocation_lng             1000163 non-null float64
3 geolocation_city            1000163 non-null object
4 geolocation_state            1000163 non-null object
dtypes: float64(2), int64(1), object(2)
memory usage: 38.2+ MB
None

```

Describe ----->

	geolocation_zip_code_prefix	geolocation_lat	geolocation_lng
count	1.000163e+06	1.000163e+06	1.000163e+06
mean	3.657417e+04	-2.117615e+01	-4.639054e+01
std	3.054934e+04	5.715866e+00	4.269748e+00
min	1.001000e+03	-3.660537e+01	-1.014668e+02
25%	1.107500e+04	-2.360355e+01	-4.857317e+01
50%	2.653000e+04	-2.291938e+01	-4.663788e+01
75%	6.350400e+04	-1.997962e+01	-4.376771e+01
max	9.999000e+04	4.506593e+01	1.211054e+02

***** Inspect Dataframe Sellers *****

Head rows ----->

	seller_id	seller_zip_code_prefix	seller_city	seller_state
0	3442f8959a84dea7ee197c632cb2df15	13023	campinas	SP
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	mogi guacu	SP
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	rio de janeiro	RJ
3	c0f3eea2e14555b6faeea3dd58c1b1c3	4195	sao paulo	SP
4	51a04a8a6bdbcb23deccc82b0b80742cf	12914	braganca paulista	SP

Information ----->

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3095 entries, 0 to 3094
Data columns (total 4 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   seller_id              3095 non-null  object
1   seller_zip_code_prefix 3095 non-null  int64
2   seller_city            3095 non-null  object
3   seller_state           3095 non-null  object
dtypes: int64(1), object(3)
memory usage: 96.8+ KB
None

```

Describe ----->

	seller_zip_code_prefix
count	3095.000000
mean	32291.059451
std	32713.453830
min	1001.000000
25%	7093.500000
50%	14940.000000
75%	64552.500000

```

max          99730.000000
***** Inspect Dataframe Products *****

Head rows ----->

   product_id  product_category_name  product_name_lenght  product_descrip
0  1e9e8ef04dbcff4541ed26657ea517e5          perfumaria          40.0
1  3aa071139cb16b67ca9e5dea641aaa2f            artes          44.0
2  96bd76ec8810374ed1b65e291975717f        esporte_lazer          46.0
3  cef67bcfe19066a932b7673e239eb23d            bebes          27.0
4  9dc1a7de274444849c219cff195d0b71  utilidades_domesticas          37.0

Information ----->

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 32951 entries, 0 to 32950
Data columns (total 9 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   product_id                           32951 non-null  object
 1   product_category_name                 32341 non-null  object
 2   product_name_lenght                   32341 non-null  float64
 3   product_description_lenght            32341 non-null  float64
 4   product_photos_qty                   32341 non-null  float64
 5   product_weight_g                      32949 non-null  float64
 6   product_length_cm                     32949 non-null  float64
 7   product_height_cm                     32949 non-null  float64
 8   product_width_cm                      32949 non-null  float64
dtypes: float64(7), object(2)
memory usage: 2.3+ MB
None

Describe ----->

   product_name_lenght  product_description_lenght  product_photos_qty  product_weight_g  pr
count          32341.000000          32341.000000          32341.000000          32949.000000
mean             48.476949             771.495285             2.188986          2276.472488
std              10.245741             635.115225             1.736766          4282.038731
min               5.000000              4.000000             1.000000           0.000000
25%              42.000000             339.000000             1.000000          300.000000
50%              51.000000             595.000000             1.000000          700.000000
75%              57.000000             972.000000             3.000000         1900.000000
max              76.000000            3992.000000            20.000000        40425.000000
***** Inspect Dataframe Category Translation *****

Head rows ----->

   product_category_name  product_category_name_english
0          beleza_saude          health_beauty
1  informatica_acessorios      computers_accessories
2          automotivo          auto
3      cama_mesa_banho          bed_bath_table
4      moveis_decoracao      furniture_decor

Information ----->

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 71 entries, 0 to 70

```

Data columns (total 2 columns):

#	Column	Non-Null Count	Dtype
0	product_category_name	71 non-null	object
1	product_category_name_english	71 non-null	object

dtypes: object(2)
memory usage: 1.2+ KB
None

Describe ----->

	product_category_name	product_category_name_english
count	71	71
unique	71	71
top	beleza_saude	health_beauty
freq	1	1

***** Inspect Dataframe Orders *****

Head rows ----->

	order_id	customer_id	order_status	order_purchase_timestamp
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-08-08 14:58:56
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-09 11:23:45
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-03-15 09:10:00
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbe7375364d82	delivered	2017-12-12 16:59:40
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-09 11:23:45

Information ----->

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 99441 entries, 0 to 99440

Data columns (total 8 columns):

#	Column	Non-Null Count	Dtype
0	order_id	99441 non-null	object
1	customer_id	99441 non-null	object
2	order_status	99441 non-null	object
3	order_purchase_timestamp	99441 non-null	object
4	order_approved_at	99281 non-null	object
5	order_delivered_carrier_date	97658 non-null	object
6	order_delivered_customer_date	96476 non-null	object
7	order_estimated_delivery_date	99441 non-null	object

dtypes: object(8)

memory usage: 6.1+ MB

None

Describe ----->

	order_id	customer_id	order_status	order_purchase_timestamp
count	99441	99441	99441	99441
unique	99441	99441	8	99441
top	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-08-08 14:58:56
freq	1	1	96478	99441

***** Inspect Dataframe Order Items *****

Head rows ----->

	order_id	order_item_id	product_id
0	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61
1	00018f77f2f0320c557190d7a144bdd3	1	e5f2d52b802189ee658865ca93d83a8f

```
2 000229ec398224ef6ca0657da4fc703e 1 c777355d18b72b67abbeef9df44fd0fd 5b51032ed
3 00024acbcd0a6daa1e931b038114c75 1 7634da152a4610f1595efa32f14722fc 9d7a1d34a5
4 00042b26cf59d7ce69dfabb4e55b4fd9 1 ac6c3623068f30de03045865e4e10089 df560393f3
```

Information ----->

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 112650 entries, 0 to 112649
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              112650 non-null object
1   order_item_id         112650 non-null int64
2   product_id            112650 non-null object
3   seller_id             112650 non-null object
4   shipping_limit_date    112650 non-null object
5   price                 112650 non-null float64
6   freight_value         112650 non-null float64
dtypes: float64(2), int64(1), object(4)
memory usage: 6.0+ MB
None
```

Describe ----->

	order_item_id	price	freight_value
count	112650.000000	112650.000000	112650.000000
mean	1.197834	120.653739	19.990320
std	0.705124	183.633928	15.806405
min	1.000000	0.850000	0.000000
25%	1.000000	39.900000	13.080000
50%	1.000000	74.990000	16.260000
75%	1.000000	134.900000	21.150000
max	21.000000	6735.000000	409.680000

***** Inspect Dataframe Payments *****

Head rows ----->

	order_id	payment_sequential	payment_type	payment_installments	pay
0	b81ef226f3fe1789b1e8b2acac839d17	1	credit_card		8
1	a9810da82917af2d9aefd1278f1dcfa0	1	credit_card		1
2	25e8ea4e93396b6fa0d3dd708e76c1bd	1	credit_card		1
3	ba78997921bbcdc1373bb41e913ab953	1	credit_card		8
4	42fdf880ba16b47b59251dd489d4441a	1	credit_card		2

Information ----->

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 103886 entries, 0 to 103885
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  -
0   order_id              103886 non-null object
1   payment_sequential    103886 non-null int64
2   payment_type          103886 non-null object
3   payment_installments  103886 non-null int64
4   payment_value         103886 non-null float64
dtypes: float64(1), int64(2), object(2)
memory usage: 4.0+ MB
None
```

Describe ----->

Task 3 :Identify primary and foreign keys

```

payment_sequential payment_installments payment_value
count      103886 000000      103886 000000      103886 000000

def identify_keys(df, primary_key, foreign_key):
    df_name = df.attrs.get('name', '')
    display(HTML(f"<b>{' '*30} Print Keys for Dataframe {df_name} {' '*30}</b>"))
    total_rows = len(df)
    unique_rows = df.drop_duplicates().shape[0]
    print('Total Rows : ', total_rows)
    print('Unique Rows : ', unique_rows)
    print('Duplicates: ', 'N' if total_rows == unique_rows else 'Y')
    # check keys
    if primary_key:
        df = df.drop_duplicates()
        if isinstance(primary_key, list) and df[primary_key].duplicated().sum() == 0:
            print('Primary Key: ', primary_key)
        elif isinstance(primary_key, str) and df[primary_key].nunique() == total_rows:
            print('Primary Key: ', primary_key)
        else:
            print('Primary Key: None')

    if foreign_key and isinstance(foreign_key, dict):
        for key, values in dict.items(foreign_key):
            print(f"Foreign Key: {key} to Dataset: {values}")
    else:
        print('Foreign Key: None')

identify_keys(df_location , ['geolocation_zip_code_prefix', 'geolocation_lat', 'geolocation_lng'])
identify_keys(df_customers , 'customer_id', {'customer_zip_code_prefix': 'location, Column- geolc
identify_keys(df_sellers , 'seller_id', {'seller_zip_code_prefix': 'location, Column- geolocatio
identify_keys(df_cat_trans , 'product_category_name', '')
identify_keys(df_products , 'product_id', {'product_category_name': 'category_translation, Colum
identify_keys(df_orders , 'order_id', {'customer_id': 'customers, Column- customer_id'})
identify_keys(df_order_item , ['order_id', 'order_item_id'], {'order_id': 'orders, Column- order_
identify_keys(df_payments , ['order_id', 'payment_sequential', 'payment_type'], {'order_id': 'ord
identify_keys(df_reviews , ['review_id', 'order_id'], {'order_id': 'orders, Column- order_id'})

0   review_id          99224 non-null object
1   order_id           99224 non-null object
2   review_score       99224 non-null int64
3   review_comment_title 11568 non-null object
4   review_comment_message 40977 non-null object
5   review_creation_date 99224 non-null object
6   review_answer_timestamp 99224 non-null object
dtypes: int64(1), object(6)
memory usage: 5.3+ MB
None

```

Describe ----->

	review_score
count	99224.000000
mean	4.086421
std	1.347579
min	1.000000
25%	4.000000
50%	5.000000
75%	5.000000
max	5.000000


```

***** Print Keys for Dataframe Location *****
Total Rows : 1000163
Unique Rows : 738332
Duplicates: Y
Primary Key: None
Foreign Key: None
***** Print Keys for Dataframe Customers *****
Total Rows : 99441
Unique Rows : 99441
Duplicates: N
Primary Key: customer_id
Foreign Key: customer_zip_code_prefix to Dataset: location, Column- geolocation_zip_code_prefix
***** Print Keys for Dataframe Sellers *****
Total Rows : 3095
Unique Rows : 3095
Duplicates: N
Primary Key: seller_id
Foreign Key: seller_zip_code_prefix to Dataset: location, Column- geolocation_zip_code_prefix
***** Print Keys for Dataframe Category Translation *****
Total Rows : 71
Unique Rows : 71
Duplicates: N
Primary Key: product_category_name
Foreign Key: None
***** Print Keys for Dataframe Products *****
Total Rows : 32951
Unique Rows : 32951
Duplicates: N
Primary Key: product_id
Foreign Key: product_category_name to Dataset: category_translation, Column- product_category_name
***** Print Keys for Dataframe Orders *****
Total Rows : 99441
Unique Rows : 99441
Duplicates: N
Primary Key: order_id
Foreign Key: customer_id to Dataset: customers, Column- customer_id
***** Print Keys for Dataframe Order Items *****
Total Rows : 112650
Unique Rows : 112650
Duplicates: N
Primary Key: ['order_id', 'order_item_id']
Foreign Key: order_id to Dataset: orders, Column- order_id
Foreign Key: product_id to Dataset: products, Column- product_id
Foreign Key: seller_id to Dataset: sellers, Column- seller_id
***** Print Keys for Dataframe Payments *****
Total Rows : 103886
Unique Rows : 103886
Duplicates: N
Primary Key: ['order_id', 'payment_sequential', 'payment_type']
Foreign Key: order_id to Dataset: orders, Column- order_id
***** Print Keys for Dataframe Reviews *****
Total Rows : 99224
Unique Rows : 99224
Duplicates: N
Primary Key: ['review_id', 'order_id']
Foreign Key: order_id to Dataset: orders, Column- order_id

```

✓ Task 4 :Understand relationships between tables

```

# current structure is liek a snowflake schema

# inside dimensions
# product dimension is connected again to category_translation dimension
# customer dimension is connected to location dimension
# sellers dimenstion is connected to location dimension

# not all dimension is connected to fact
# product is connected to fact(order) via order_item
# seller is connected to fact(order) via order_item

```

```
# so we merge everything into a single dataframe and make it a star schema equivalent in ML
```

✓ Step 2: Data Cleaning and Preprocessing

Task 1: Handle missing values appropriately

```
# check missing values
def check_missing_values(df):
    df_name = df.attrs.get('name', '')
    display(HTML(f"<b>{' '*30} Check Missing Values for Dataframe {df_name} {' '*30}</b>"))
    print('** Check for Nan----->')
    print(df.isna().sum())
    check_empty = (df.select_dtypes(include='object')
                    .apply(lambda x: x.str.strip())
                    .eq('')
                    .sum())
    print('\n** Check for Empty String----->')
    print(check_empty)
```

```
check_missing_values(df_customers) # no missing value
check_missing_values(df_location) # no missing value
check_missing_values(df_sellers) # no missing value
check_missing_values(df_products) # missing value
check_missing_values(df_cat_trans) # no missing value

check_missing_values(df_orders) # missing value
check_missing_values(df_order_item) # no missing value
check_missing_values(df_payments) # no missing value
check_missing_values(df_reviews) # missing value
```



```

***** Check Missing Values for Dataframe Customers *****
** Check for Nan----->
customer_id          0
customer_unique_id   0
customer_zip_code_prefix  0
customer_city        0
customer_state       0
dtype: int64

** Check for Empty String----->
customer_id          0
customer_unique_id   0
customer_city        0
customer_state       0
dtype: int64
***** Check Missing Values for Dataframe Location *****
** Check for Nan----->
geolocation_zip_code_prefix  0
geolocation_lat              0
geolocation_lng              0
geolocation_city             0
geolocation_state            0
dtype: int64

** Check for Empty String----->
geolocation_city      0
geolocation_state     0
dtype: int64
***** Check Missing Values for Dataframe Sellers *****
** Check for Nan----->
seller_id          0
seller_zip_code_prefix  0
seller_city        0
seller_state       0
dtype: int64

** Check for Empty String----->
seller_id      0
seller_city    0
seller_state   0
dtype: int64
***** Check Missing Values for Dataframe Products *****
** Check for Nan----->
product_id          0
product_category_name  610
product_name_lenght   610
product_description_lenght  610
product_photos_qty     610
product_weight_g       2
product_length_cm      2
product_height_cm      2
product_width_cm       2
dtype: int64

** Check for Empty String----->
product_id          0
product_category_name  0
dtype: int64
***** Check Missing Values for Dataframe Category Translation *****
** Check for Nan----->
product_category_name          0
product_category_name_english  0
dtype: int64

** Check for Empty String----->
product_category_name          0
product_category_name_english  0
dtype: int64

```

✓ Handle Missing Values in Products

```

***** Check Missing Values for Dataframe Orders *****
** Check for Nan----->

```

```

# drop data where very small number of missing values
df_products = df_products.dropna(subset=['product_weight_g', 'product_length_cm', 'product_width_cm'])

```

```
# product_category_name is important categoricl value and we will fill with unknown
df_products['product_category_name'] = df_products['product_category_name'].fillna('unknown')

# product_photos_qty is number, we can fill with 0
df_products['product_photos_qty'] = df_products['product_photos_qty'].fillna(0)

# product_name_lenght and product_description_lenght are derived column, for unknown categories v
# if it was random, we could have used mean/median
df_products['product_name_lenght'] = df_products['product_name_lenght'].fillna(0)
df_products['product_description_lenght'] = df_products['product_description_lenght'].fillna(0)
```

```
order_delivered_carrier_date 0
check_missing_values(df_products) # missing value
```

```
dtype: int64
***** Check Missing Values for Dataframe Products *****
***** Check Missing Values for Dataframe Order Items *****
** Check for Nan----->
product_id 0
order_id 0
product_category_name 0 0
order_item_id 0 0
product_name_lenght 0 0
product_id 0
product_description_lenght 0
seller_id 0
product_photos_qty 0 0
shipping_limit_date 0 0
product_weight_g 0 0
price 0
product_length_cm 0 0
freight_value 0 0
product_height_cm 0 0
product_width_cm 0
dtype: int64
** Check for Empty String----->
order_id 0
product_id 0 0
seller_id 0 0
product_category_name 0 0
shipping_limit_date 0
dtype: int64
```

```
***** Check Missing Values for Dataframe Payments *****
```

```
** Check for Nan----->
```

✓ Handle Missing Values in Orders

```
payment_sequential 0
payment_type 0
```

```
print('Length: ', df_orders.shape[0])
check_missing_values(df_orders) # missing value
# we cannot drop this columns as missing value means order not approved/ not delivered
# we can create flags new features
df_orders['is_approved'] = df_orders['order_approved_at'].notnull()
df_orders['is_shipped'] = df_orders['order_delivered_carrier_date'].notnull()
df_orders['is_delivered'] = df_orders['order_delivered_customer_date'].notnull()
```

```
Length: 9941
review_id
***** Check Missing Values for Dataframe Orders *****
** Check for Nan----->
order_id 0
review_id 0
review_score 0 0
review_comment_title 87656 0
review_comment_message 58247 0
review_comment_timestamp 0 0
review_approved_at 0 160
review_approved_at_timestamp 0 160
dtype: int64
order_delivered_carrier_date 1783
order_delivered_customer_date 2965
order_estimated_delivery_date--> 0
dtype: int64
review_id 0
order_id 0
review_comment_title string--2-->
review_comment_message 27 0
review_creation_date 0 0
review_answer_timestamp 0 0
dtype: int64
base_timestamp 0
order_approved_at 0
order_delivered_carrier_date 0
order_delivered_customer_date 0
order_estimated_delivery_date 0
dtype: int64
```

Handle Missing Values in Reviews

```
# reviews are optional, so we have lots of missing values
# replace empty strings
df_reviews['review_comment_title'] = df_reviews['review_comment_title'].str.strip().replace('', '')
df_reviews['review_comment_message'] = df_reviews['review_comment_message'].str.strip().replace('', '')

# we cannot put mode for random reviews, we create new feature
df_reviews['has_comments'] = df_reviews['review_comment_title'].notnull()
df_reviews['has_message'] = df_reviews['review_comment_message'].notnull()
```

```
check_missing_values(df_reviews) # missing value
```

```
***** Check Missing Values for Dataframe Reviews *****
** Check for Nan----->
review_id          0
order_id           0
review_score       0
review_comment_title      87658
review_comment_message    58274
review_creation_date      0
review_answer_timestamp  0
has_comments        0
has_message         0
dtype: int64

** Check for Empty String----->
review_id          0
order_id           0
review_comment_title      0
review_comment_message    0
review_creation_date      0
review_answer_timestamp  0
dtype: int64
```

Task 2: Remove duplicate records

```
# row level duplicates
def duplicate_check(df):
    df_name = df.attrs.get('name', '')
    display(HTML(f"<b>{' '*3} Check Duplicate Values for Dataframe {df_name} {' '*3}</b>"))
    has_duplicate = ('Yes' if df.shape[0] != df.drop_duplicates().shape[0]
                    else 'No')
    print('Duplicate: ', has_duplicate)
    print("\n")

duplicate_check(df_customers)
duplicate_check(df_location)
duplicate_check(df_sellers)
duplicate_check(df_products)
duplicate_check(df_cat_trans)

duplicate_check(df_orders)
duplicate_check(df_order_item)
duplicate_check(df_payments)
duplicate_check(df_reviews)
```

*** Check Duplicate Values for Dataframe Customers ***

Duplicate: No

*** Check Duplicate Values for Dataframe Location ***

Duplicate: Yes

*** Check Duplicate Values for Dataframe Sellers ***

Duplicate: No

*** Check Duplicate Values for Dataframe Products ***

Duplicate: No

*** Check Duplicate Values for Dataframe Category Translation ***

Duplicate: No

*** Check Duplicate Values for Dataframe Orders ***

Duplicate: No

*** Check Duplicate Values for Dataframe Order Items ***

Duplicate: No

*** Check Duplicate Values for Dataframe Payments ***

Duplicate: No

*** Check Duplicate Values for Dataframe Reviews ***

Duplicate: No

✓ df_location has duplicates at row level, we have to remove it

```
import unicodedata

# check for key level duplicates and clean
df_location = df_location.drop_duplicates()
duplicate_check(df_location)

print(list(df_location['geolocation_city'].unique())[0:100])
# has some characters which are unwanted, need to clean text
def clean_text(text):
    if text == '':
        return text
    text = text.lower().strip()
    text = unicodedata.normalize('NFKD', text)
    text = text.encode('ascii', 'ignore').decode('utf-8')
    text = text.replace('-', ' ')
    text = ' '.join(text.split())
    return text

df_location['geolocation_city'] = df_location['geolocation_city'].apply(clean_text)
```

*** Check Duplicate Values for Dataframe Location ***

Duplicate: No

['sao paulo', 'são paulo', 'sao bernardo do campo', 'jundiaí', 'taboão da serra', 'sãopaulo', 'sp'

```
print(list(df_location['geolocation_city'].unique())[0:100])
```

```
['sao paulo', 'sao bernardo do campo', 'jundiai', 'taboao da serra', 'saopaulo', 'sp', 'sao jose d
```

✓ Task 3: Convert date columns to datetime format

required for time analysis (monthly trends, delays, etc.) and Avoids string comparison error

```
def convert_date(df, cols):
    for col in cols:
        df[col] = pd.to_datetime(df[col], errors = 'coerce')

convert_date(df_orders, ['order_purchase_timestamp', 'order_approved_at', 'order_delivered_carrier_
                        'order_delivered_customer_date', 'order_estimated_delivery_c
convert_date(df_order_item, ['shipping_limit_date'])
convert_date(df_reviews, ['review_creation_date', 'review_answer_timestamp'])
```

✓ Task 4 : Validate data types and ranges

```
df_customers['customer_zip_code_prefix'] = df_customers['customer_zip_code_prefix'].astype(str) #
df_customers.dtypes
```

```
customer_id           object
customer_unique_id    object
customer_zip_code_prefix  object
customer_city         object
customer_state        object
dtype: object
```

```
df_location.dtypes
```

```
geolocation_zip_code_prefix    int64
geolocation_lat                float64
geolocation_lng                float64
geolocation_city               object
geolocation_state              object
dtype: object
```

```
df_location['geolocation_zip_code_prefix'] = df_location['geolocation_zip_code_prefix'].astype(st

#range validations
df_location['geolocation_lat'].between(-90, 90).all()
df_location['geolocation_lng'].between(-180, 180).all()
```

```
np.True_
```

```
df_sellers.dtypes
```

```
seller_id           object
seller_zip_code_prefix  int64
seller_city         object
seller_state        object
dtype: object
```

```
df_sellers['seller_zip_code_prefix'] = df_sellers['seller_zip_code_prefix'].astype(str) # to pres
```

```
df_products.dtypes
```

```
product_id           object
product_category_name  object
product_name_lenght   float64
```



```
product_description_lenght    float64
product_photos_qty           float64
product_weight_g              float64
product_length_cm             float64
product_height_cm             float64
product_width_cm              float64
dtype: object
```

```
# product_name_lenght, product_description_lenght, product_photos_qty should be number
cols = ['product_name_lenght', 'product_description_lenght', 'product_photos_qty']

for col in cols:
    df_products[col] = df_products[col].astype(int)

# range validarions
# the weight and height should not be negative
weight_height_cols = ['product_weight_g', 'product_length_cm', 'product_height_cm', 'product_width_cm']

(df_products[weight_height_cols] >= 0).all().all()

np.True_
```

```
df_products.head()
```

	product_id	product_category_name	product_name_lenght	product_description_lenght
0	1e9e8ef04dbcff4541ed26657ea517e5	perfumaria	40	10
1	3aa071139cb16b67ca9e5dea641aaa2f	artes	44	10
2	96bd76ec8810374ed1b65e291975717f	esporte_lazer	46	10
3	cef67bcfe19066a932b7673e239eb23d	bebes	27	10
4	9dc1a7de274444849c219cff195d0b71	utilidades_domesticas	37	10

```
df_cat_trans.dtypes
```

```
product_category_name      object
product_category_name_english  object
dtype: object
```

```
# check if produc category is unique
```

```
df_cat_trans['product_category_name'].nunique() == len(df_cat_trans)
```

```
True
```

```
df_orders.dtypes
```

```
order_id                object
customer_id             object
order_status            object
order_purchase_timestamp  datetime64[ns]
order_approved_at        datetime64[ns]
order_delivered_carrier_date  datetime64[ns]
order_delivered_customer_date  datetime64[ns]
order_estimated_delivery_date  datetime64[ns]
is_approved             bool
is_shipped              bool
is_delivered            bool
dtype: object
```

```
# order status should be categorical
```

```
df_orders['order_status'].unique()
```

```
array(['delivered', 'invoiced', 'shipped', 'processing', 'unavailable',  
      'canceled', 'created', 'approved'], dtype=object)
```

```
(df_orders['order_purchase_timestamp'] > df_orders['order_approved_at']).any()
```

```
np.False_
```

```
df_orders['order_status'] = df_orders['order_status'].astype('category')
```

```
df_orders.dtypes
```

```
order_id                object  
customer_id             object  
order_status            category  
order_purchase_timestamp  datetime64[ns]  
order_approved_at        datetime64[ns]  
order_delivered_carrier_date  datetime64[ns]  
order_delivered_customer_date  datetime64[ns]  
order_estimated_delivery_date  datetime64[ns]  
is_approved              bool  
is_shipped                bool  
is_delivered              bool  
dtype: object
```

```
# price and freight >= 0
```

```
(df_order_item['price'] > 0).all()
```

```
np.True_
```

```
(df_order_item['freight_value'] >= 0).all()
```

```
np.True_
```

```
df_order_item.duplicated(['order_id', 'order_item_id']).sum()
```

```
np.int64(0)
```

```
df_order_item['price'].describe()
```

```
count    112650.000000  
mean      120.653739  
std       183.633928  
min        0.850000  
25%       39.900000  
50%       74.990000  
75%      134.900000  
max      6735.000000  
Name: price, dtype: float64
```

```
df_payments.dtypes
```

```
order_id                object  
payment_sequential       int64  
payment_type             object  
payment_installments     int64  
payment_value            float64  
dtype: object
```

```
df_payments['payment_type'].unique()
```

```
array(['credit_card', 'boleto', 'voucher', 'debit_card', 'not_defined'],
      dtype=object)
```

```
df_payments['payment_type'] = df_payments['payment_type'].astype('category') # payment type should be category
(df_payments['payment_value'] < 0).any() # payment value less than zero
```

```
np.False_
```

```
df_payments.duplicated(['order_id', 'payment_sequential']).sum()
```

```
np.int64(0)
```

```
df_payments[['order_id', 'payment_sequential']].drop_duplicates().shape[0] == len(df_payments)
```

```
True
```

```
df_payments['payment_type'] = df_payments['payment_type'].cat.rename_categories({
    'not_defined': 'unknown'
})
```

```
df_reviews.dtypes
```

```
review_id          object
order_id           object
review_score       int64
review_comment_title object
review_comment_message object
review_creation_date datetime64[ns]
review_answer_timestamp datetime64[ns]
has_comments       bool
has_message        bool
dtype: object
```

```
df_reviews['review_score'].between(1, 5).all()
```

```
np.True_
```

```
(df_reviews['review_answer_timestamp'] >= df_reviews['review_creation_date']).all()
```

```
np.True_
```

✓ • Standardize column names if required

```
df_customers.columns
```

```
Index(['customer_id', 'customer_unique_id', 'customer_zip_code_prefix',
      'customer_city', 'customer_state'],
      dtype='object')
```

```
def standardize_column(df):
    df.columns = (
        df.columns.str.strip()
        .str.lower()
        .str.replace(' ', '_')
        .str.replace('-', '_')
    )
    return df
```

```
df_customers = standardize_column(df_customers)
df_location = standardize_column(df_location)
df_sellers = standardize_column(df_sellers)
```

```
df_products = standardize_column(df_products)
df_cat_trans = standardize_column(df_cat_trans)
df_orders = standardize_column(df_orders)
df_order_item = standardize_column(df_order_item)
df_payments = standardize_column(df_payments)
df_reviews = standardize_column(df_reviews)
```

✓ Step 3: Data Integration (Critical Component)

You must construct a Master Dataset by merging multiple tables Recommended sequence:

1. orders + customers
2. orders + order_items
3. order_items + products
4. orders + payments
5. orders + reviews
6. order_items + sellers
7. products + category_translation

```
# orders + customers
df_orders_customers = df_orders.merge(df_customers, on='customer_id', how='left')
len(df_orders), len(df_customers), len(df_orders_customers)
```

(99441, 99441, 99441)

```
# orders + order_items
df_order_item_merged = df_orders_customers.merge(df_order_item, on='order_id', how='left')
len(df_orders_customers), len(df_order_item), len(df_order_item_merged)
```

(99441, 112650, 113425)

```
# order_items + products
df_order_item_products = df_order_item_merged.merge(df_products, on="product_id", how = 'left')
len(df_order_item_merged), len(df_products), len(df_order_item_products)
```

(113425, 32949, 113425)

```
# orders + payments
print(df_payments['order_id'].nunique(), len(df_payments))
df_payments.head()
```

99440 103886

		order_id	payment_sequential	payment_type	payment_installments	payment_value
0	b81ef226f3fe1789b1e8b2acac839d17		1	credit_card	8	
1	a9810da82917af2d9aefd1278f1dcfa0		1	credit_card	1	
2	25e8ea4e93396b6fa0d3dd708e76c1bd		1	credit_card	1	
3	ba78997921bbcdc1373bb41e913ab953		1	credit_card	8	
4	42fd880ba16b47b59251dd489d4441a		1	credit_card	2	

```
df_payment_agg = df_payments.groupby('order_id').agg({
    'payment_value': 'sum',
    'payment_installments': 'max'
}).reset_index()
```

```
df_payment_agg.head()
```

	order_id	payment_value	payment_installments
0	00010242fe8c5a6d1ba2dd792cb16214	72.19	2
1	00018f77f2f0320c557190d7a144bdd3	259.83	3
2	000229ec398224ef6ca0657da4fc703e	216.87	5
3	00024acbcd0a6daa1e931b038114c75	25.78	2
4	00042b26cf59d7ce69dfabb4e55b4fd9	218.04	3

```
df_order_payments = df_order_item_products.merge(df_payment_agg, on="order_id", how = 'left')
len(df_order_item_products), len(df_payment_agg), len(df_order_payments)
```

```
(113425, 99440, 113425)
```

```
# orders + reviews
df_reviews.duplicated('order_id').sum()
```

```
np.int64(551)
```

```
# number of rows should not increase after adding reviews, this means
# we have more than one review for some orders
df_reviews[df_reviews.duplicated('order_id', keep=False)].sort_values('order_id').head()
```

	review_id	order_id	review_score	review_c
25612	89a02c45c340aeeb1354a24e7d4b2c1e	0035246a40f520710769010f752e7507	5	
22423	2a74b0559eb58fc1ff842ecc999594cb	0035246a40f520710769010f752e7507	5	
22779	ab30810c29da5da8045216f0f62652a2	013056cfe49763c6f66bda03396c5ee3	5	
68633	73413b847f63e02bc752b364f6d05ee9	013056cfe49763c6f66bda03396c5ee3	4	
854	830636803620cdf8b6ffaf1b2f6e92b2	0176a6846bcb3b0d3aa3116a9a768597	5	

```
df_reviews_agg = df_reviews.groupby('order_id').agg({
    'review_score': 'mean',
    'has_message': 'max',
    'has_comments': 'max'
}).reset_index()
```

```
df_order_reviews = df_order_payments.merge(df_reviews_agg, on="order_id", how = 'left')
len(df_order_payments), len(df_reviews_agg), len(df_order_reviews)
```

```
(113425, 98673, 113425)
```

```
# order_items + sellers
```

```
df_order_sellers = df_order_reviews.merge(df_sellers, on="seller_id", how = 'left')
len(df_order_reviews), len(df_sellers), len(df_order_sellers)
```

```
(113425, 3095, 113425)
```

```
# products + category_translation
df_master = df_order_sellers.merge(df_cat_trans, on="product_category_name", how = 'left')
len(df_order_sellers), len(df_cat_trans), len(df_master)
```

```
(113425, 71, 113425)
```

✓ Step 4: Feature Engineering

✓ • Total order value (aggregated from order_items or payments)

```
df_order_item['item_total'] = df_order_item['price'] + df_order_item['freight_value']

df_total_order_value = df_order_item.groupby('order_id')['item_total'].sum().reset_index()
df_total_order_value.head()

df_master = df_master.merge(df_total_order_value, on="order_id", how = "left")
df_master.head()
```

	order_id	customer_id	order_status	order_purchase_timestamp
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-11-01
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-01
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01-01
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-01
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-01

5 rows × 5 columns

✓ • Delivery time (order purchase to delivery date)

```
df_master['order_purchase_timestamp'].isna().sum()
df_master['order_delivered_customer_date'].isna().sum() # to be handled for orders not yet delivered

df_master['delivery_time'] = (df_master['order_delivered_customer_date'] - df_master['order_purchase_timestamp']).dt.days
```

```
df_master['delivery_time'].isna().sum()
```

```
np.int64(3229)
```

```
df_master['delivery_time'] = df_master['delivery_time'].fillna(-1)
```

```
(df_master['delivery_time'] == 0).sum()
```

```
np.int64(18)
```

✓ • Number of items per order

```
df_order_item.head()
df_item_per_order = df_order_item.groupby('order_id')['order_item_id'].size().reset_index(name='item_count')
df_item_per_order.head()
df_item_per_order['item_count'].isna().sum()
```

```
np.int64(0)
```

```
df_master = pd.merge(df_master, df_item_per_order, on="order_id", how="left")
df_master.head()
```

	order_id	customer_id	order_status	order_purchase_date
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-01
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-01
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01-01
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-10-01
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-01

5 rows × 5 columns

```
# why item count is coming as 1.0
df_master['item_count'].isna().sum()
```

```
np.int64(775)
```

```
df_master['item_count'] = df_master['item_count'].fillna(0)
df_master['item_count'] = df_master['item_count'].astype(int)
df_master['item_count'].isna().sum()
```

```
np.int64(0)
```

```
df_master.head()
```

	order_id	customer_id	order_status	order_purchase_date	item_count
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-01	1
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-01	1
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01-01	1
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-10-01	1
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-01	1

5 rows × 6 columns

• Customer purchase frequency

```
df_orders['customer_id'].nunique()
```

```
99441
```

Start coding or [generate](#) with AI.

```
len(df_orders)
```

```
99441
```

```
# there are one order per customer, so we simulate some repeat customers
```

```
import numpy as np

# copy to avoid messing original
df_orders_backup = df_orders.copy()

# choose % of rows to modify
pct = 0.3 # 30% orders will be reassigned
n = int(len(df_orders) * pct)

# pick random indices to modify
idx = np.random.choice(df_orders.index, size=n, replace=False)

# pick random customer_ids (from existing pool)
random_customers = np.random.choice(
    df_orders['customer_id'],
    size=n,
    replace=True
)

# assign
df_orders.loc[idx, 'customer_id'] = random_customers
```

```
# to check loyal/repeat customers
df_orders.head()
any(df_orders['customer_id'].isna())
```

```
False
```

```
df_customer_purchase_frequency = df_orders.groupby("customer_id").size().reset_index(name= 'purchase_frequency')
df_customer_purchase_frequency['purchase_frequency'] = df_customer_purchase_frequency['purchase_frequency'] * pct
```

```
#df_master = df_master.drop(columns=['purchase_frequency_x', 'purchase_frequency_y', 'purchase_frequency_z'])
df_master.head()
```

	order_id	customer_id	order_status	order_purchase_frequency
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-01
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-01
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01-01
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-10-01
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-01

5 rows × 41 columns

```
df_master = pd.merge(df_master, df_customer_purchase_frequency, on="customer_id", how = "left")
```



```
df_master['purchase_frequency'] = df_master['purchase_frequency'].fillna(0).astype(int)
df_master.head()
```

	order_id	customer_id	order_status	order_purchas
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-10
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01

5 rows × 42 columns

```
any(df_master['purchase_frequency'].isna())
```

```
False
```

✓ • Customer lifetime value (basic approximation)

```
# high CLV means valuable customer
# get sum at order item level, per order sum price, sum freight
df_order_value = df_order_item.groupby('order_id').agg({'price': 'sum', 'freight_value': 'sum'})
any(df_order_value['price'].isna())
df_order_value['freight_value'].isna().sum()
# total = sum_price + sum_freight
df_order_value['total'] = df_order_value['price'] + df_order_value['freight_value']

# merge with order, get customer id
df_order_value = (pd.merge(df_orders[['order_id', 'customer_id']],
                          df_order_value,
                          on='order_id',
                          how='left'))
# apply group by customer id, take sum
df_order_value = df_order_value.groupby('customer_id')['total'].sum().reset_index(name='customer_
df_order_value['customer_lifetime_value'].isna().sum()

np.int64(0)
```

```
# merge with master
df_master = pd.merge(df_master, df_order_value, on='customer_id', how='left')
```

```
df_master.head(2)
```

	order_id	customer_id	order_status	order_purchas
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01

2 rows × 43 columns

✓ • Average order value per customer

```

df_order_value = df_order_item.groupby('order_id').agg({'price': 'sum', 'freight_value': 'sum'}).
any(df_order_value['price'].isna())
df_order_value['freight_value'].isna().sum()
# total = sum_price + sum_freight
df_order_value['total'] = df_order_value['price'] + df_order_value['freight_value']

# merge with order, get customer id
df_order_value = (pd.merge(df_orders[['order_id', 'customer_id']],
                           df_order_value,
                           on='order_id',
                           how='left'))
any(df_order_value['total'].isna())
df_order_value['total'] = df_order_value['total'].fillna(0)
df_order_value = df_order_value.groupby('customer_id')['total'].mean().reset_index(name = 'avg_order_value')
df_order_value.head(2)

```

	customer_id	avg_order_value
0	00012a2ce6f8dcda20d059ce98491703	114.74
1	000161a058600d5901f007fab4c27140	67.41

```

# merge with master
df_master = pd.merge(df_master, df_order_value, on='customer_id', how='left')

```

✓ Step 5: Exploratory Data Analysis (EDA)

Perform structured analysis across the following dimensions:

✓ Customer Analysis

- New vs repeat customers
- High-value vs low-value customers
- Geographic distribution of customers

```
df_master.head()
```

	order_id	customer_id	order_status	order_purchase_date
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-01
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-01-01
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-01-01
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-10-01
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-01-01

5 rows × 44 columns

```
len(df_orders)
```

99441

```
df_orders['customer_id'].nunique()
```

```
77376
```

```
df_customer_order = df_orders.groupby('customer_id').size().reset_index(name= 'num_orders')
```

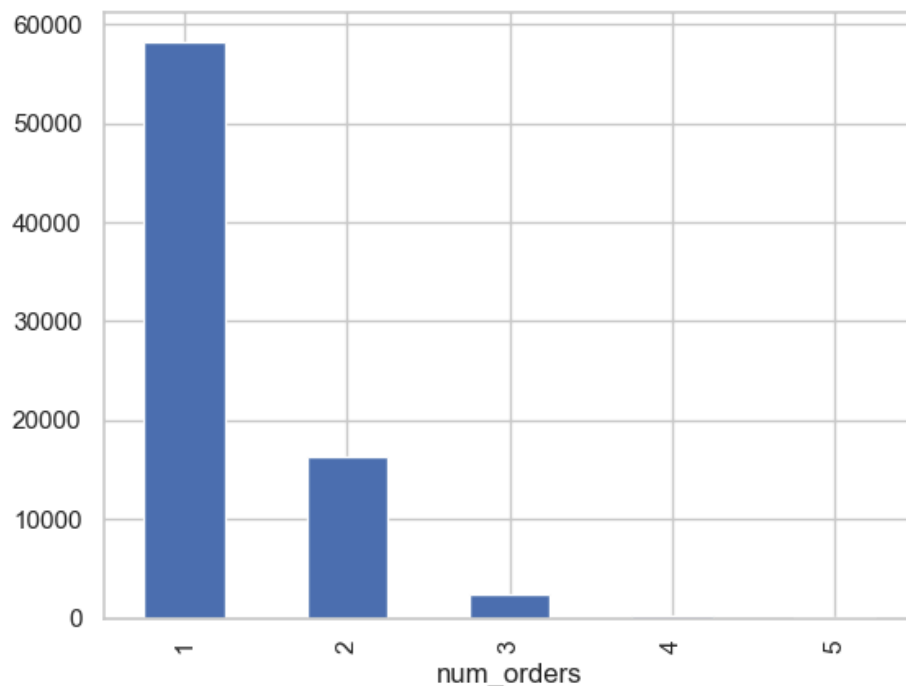
```
df_customer_order.head()
```

	customer_id	num_orders
0	00012a2ce6f8dcda20d059ce98491703	1
1	000161a058600d5901f007fab4c27140	1
2	0001fd6190edaaf884bcdf3d49edf079	1
3	0002414f95344307404f0ace7a26f1d5	2
4	000379cdec625522490c315e70c7a9fb	1

```
df_customer_order['customer_type'] = df_customer_order['num_orders'].apply(lambda x: 'repeat' if
```

```
df_customer_order['num_orders'].value_counts().plot(kind='bar')
```

<Axes: xlabel='num_orders'>



```
df_customer_order.head(1)
```

	customer_id	num_orders	customer_type
0	00012a2ce6f8dcda20d059ce98491703	1	new

```
# merge into master
```

```
df_master = pd.merge(df_master, df_customer_order[['customer_id', 'customer_type']], on='customer
```

```
# check different behavior
```

```
# compare revenue
```

```
df_master.groupby('customer_type')['item_total'].mean()
```

customer_type	
new	183.040462

```
repeat    175.853538
Name: item_total, dtype: float64
```

```
df_master.columns
```

```
Index(['order_id', 'customer_id', 'order_status', 'order_purchase_timestamp',
      'order_approved_at', 'order_delivered_carrier_date',
      'order_delivered_customer_date', 'order_estimated_delivery_date',
      'is_approved', 'is_shipped', 'is_delivered', 'customer_unique_id',
      'customer_zip_code_prefix', 'customer_city', 'customer_state',
      'order_item_id', 'product_id', 'seller_id', 'shipping_limit_date',
      'price', 'freight_value', 'product_category_name',
      'product_name_lenght', 'product_description_lenght',
      'product_photos_qty', 'product_weight_g', 'product_length_cm',
      'product_height_cm', 'product_width_cm', 'payment_value',
      'payment_installments', 'review_score', 'has_message', 'has_comments',
      'seller_zip_code_prefix', 'seller_city', 'seller_state',
      'product_category_name_english', 'item_total', 'delivery_time',
      'item_count', 'purchase_frequency', 'customer_lifetime_value',
      'avg_order_value', 'customer_type'],
      dtype='object')
```

```
# compare order count
df_master.groupby('customer_type')['item_count'].mean()
```

```
customer_type
new         1.385523
repeat      1.395900
Name: item_count, dtype: float64
```

```
# compare review
df_master.groupby('customer_type')['review_score'].mean()
```

```
customer_type
new         4.009581
repeat      4.031199
Name: review_score, dtype: float64
```

✓ EDA • High-value vs low-value customers

```
df_master['customer_lifetime_value'].isna().sum()
```

```
np.int64(25143)
```

```
df_master['customer_lifetime_value'].count()
```

```
np.int64(88282)
```

```
df_master['customer_lifetime_value'] = df_master['customer_lifetime_value'].fillna(0)
```

```
threshold = df_master['customer_lifetime_value'].quantile(0.75)
threshold
```

```
np.float64(212.83)
```

```
df_master['high_value'] = df_master['customer_lifetime_value'].apply(lambda x: 'high' if x > thre
```

```
df_master['high_value'].value_counts()
```

```
high_value
low      85072
high     28353
Name: count, dtype: int64
```

```
# check revenue for each segment
df_master.groupby('high_value')['customer_lifetime_value'].sum()
```

```
high_value
high      13471213.08
low       6231049.91
Name: customer_lifetime_value, dtype: float64
```

```
# check for order count
df_master.groupby('high_value')['item_count'].mean()
```

```
high_value
high      1.755052
low       1.263177
Name: item_count, dtype: float64
```

```
df_master.groupby('high_value')['item_total'].mean()
```

```
high_value
high      340.767300
low       126.355132
Name: item_total, dtype: float64
```

```
df_master.groupby('high_value')['purchase_frequency'].mean()
```

```
high_value
high      1.605474
low       0.799123
Name: purchase_frequency, dtype: float64
```

• Geographic distribution of customers

```
df_master['order_item_total'] = df_master['price'] + df_master['freight_value']
```

```
# check with geography has more orders, customers, delivery time etx
```

```
df_geo = df_master.groupby('customer_state').agg({
    'customer_id': 'nunique'
    , 'order_item_total': 'sum'
    , 'delivery_time': 'mean'
}).reset_index() \
    .sort_values(by='order_item_total', ascending=False)
```

```
df_geo = df_geo.rename(columns={
    'customer_id': 'customer_count',
    'order_item_total' : 'total_revenue'
})
```

```
df_geo.head(5)
```

	customer_state	customer_count	total_revenue	delivery_time
25	SP	41746	5921678.12	7.992974
18	RJ	12852	2129681.98	14.130002
10	MG	11635	1856161.49	11.228669
22	RS	5466	885826.76	14.367523
17	PR	5045	800935.44	11.183169

```
# so we see which state generates more revenue
# and we also check which state has more delivery time
```

EDA - Revenue and Order Analysis

• Monthly revenue trends

```
# create month-year column
df_master['order_month'] = df_master['order_purchase_timestamp'].dt.to_period('M')
```

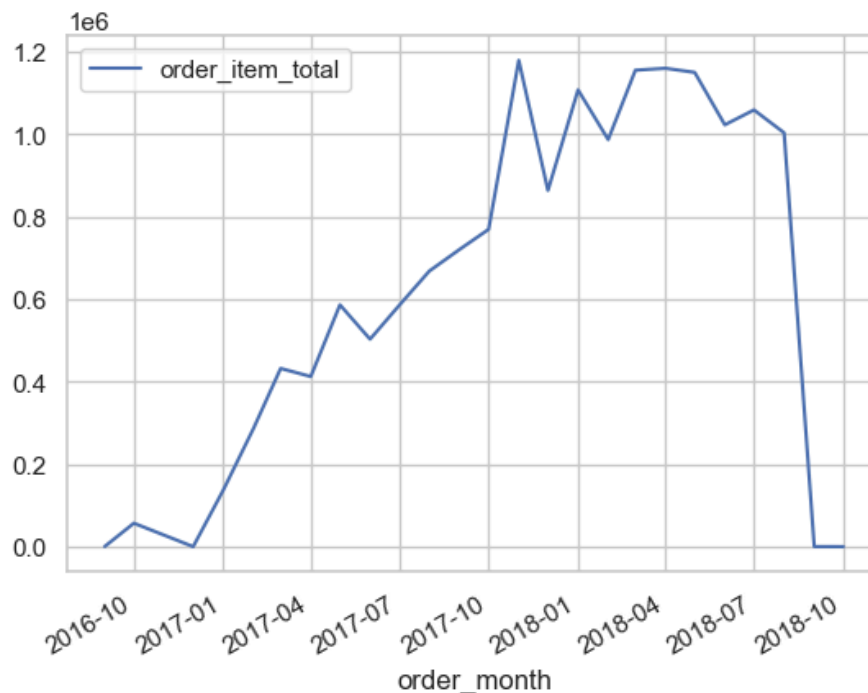
```
df_monthly = df_master.groupby('order_month')['order_item_total'].sum().reset_index()
```

```
df_monthly.head()
```

	order_month	order_item_total
0	2016-09	354.75
1	2016-10	56808.84
2	2016-12	19.62
3	2017-01	137188.49
4	2017-02	286280.62

```
df_monthly['order_month'] = df_monthly['order_month'].dt.to_timestamp()
df_monthly.plot(x='order_month', y='order_item_total', kind='line')
```

<Axes: xlabel='order_month'>



• Order volume trends

```
# monthly order count
df_monthly_orders = df_master.groupby('order_month')['order_id'].nunique().reset_index(name='num_
```

```
df_monthly_orders.head()
```

	order_month	num_order
0	2016-09	4
1	2016-10	324
2	2016-12	1
3	2017-01	800
4	2017-02	1780

```
df_monthly_orders.plot(x='order_month', y='num_order', kind='line')
# order volume increase over time
```

<Axes: xlabel='order_month'>



✓ • Peak sales periods

```
df_monthly.columns
```

```
Index(['order_month', 'order_item_total'], dtype='object')
```

```
threshold = df_monthly['order_item_total'].quantile(.75)
```

```
# top 25%
df_monthly[df_monthly['order_item_total'] > threshold] \
.sort_values(by='order_item_total', ascending=False)
```

	order_month	order_item_total
13	2017-11-01	1179143.77
18	2018-04-01	1159698.04
17	2018-03-01	1155126.82
19	2018-05-01	1149781.82
15	2018-01-01	1107301.89
21	2018-07-01	1058728.03

```
# peak sales periods
df_monthly['order_item_total'].idxmax()
df_monthly.loc[df_monthly['order_item_total'].idxmax()]
```

```
order_month      2017-11-01 00:00:00
order_item_total      1179143.77
Name: 13, dtype: object
```

```
# top 3 monthly sales
df_monthly.sort_values(by='order_item_total',ascending=False).head(3)
```

	order_month	order_item_total
13	2017-11-01	1179143.77
18	2018-04-01	1159698.04
17	2018-03-01	1155126.82

Product Analysis

- Top-selling product categories
- Revenue contribution by category
- Product demand distribution

✓ • Top-selling product categories

```
df_master['product_category_name_english'].value_counts().head(5)
```

```
product_category_name_english
bed_bath_table      11115
health_beauty       9670
sports_leisure      8641
furniture_decor     8334
computers_accessories 7827
Name: count, dtype: int64
```

```
df_master['product_category_name_english'].nunique()
```

```
71
```

✓ • Revenue contribution by category

```
df_product_volume = df_master.groupby('product_category_name_english')[['order_item_total']].sum()
```



```
# top 5
df_product_volume.sort_values(by='order_item_total', ascending=False)
```

	order_item_total
product_category_name_english	
health_beauty	1441248.07
watches_gifts	1305541.61
bed_bath_table	1241681.72
sports_leisure	1156656.48
computers_accessories	1059272.40
...	...
flowers	1598.91
home_comfort_2	1170.58
cds_dvds_musicals	954.99
fashion_childrens_clothes	665.36
security_and_services	324.51

71 rows × 1 columns

• Product demand distribution

```
df_demand = df_master['product_category_name_english'].value_counts().reset_index(name='order_item_count')
```

```
df_demand['percentage'] = df_demand['order_item_count']/df_demand['order_item_count'].sum()
```

```
df_demand.head(10)
```

	product_category_name_english	order_item_count	percentage
0	bed_bath_table	11115	0.100115
1	health_beauty	9670	0.087100
2	sports_leisure	8641	0.077831
3	furniture_decor	8334	0.075066
4	computers_accessories	7827	0.070500
5	housewares	6964	0.062726
6	watches_gifts	5991	0.053962
7	telephony	4545	0.040938
8	garden_tools	4347	0.039154
9	auto	4235	0.038146

• Seller Analysis

- Top-performing sellers
- Seller contribution to revenue
- Seller distribution

✓ Top-performing sellers

```
df_sellers.head(5)
```

	seller_id	seller_zip_code_prefix	seller_city	seller_state
0	3442f8959a84dea7ee197c632cb2df15	13023	campinas	SP
1	d1b65fc7debc3361ea86b5f14c68d2e2	13844	mogi guacu	SP
2	ce3ad9de960102d0677a81f5d0bb7b2d	20031	rio de janeiro	RJ
3	c0f3eea2e14555b6faeea3dd58c1b1c3	4195	sao paulo	SP
4	51a04a8a6bdbcb23deccc82b0b80742cf	12914	braganca paulista	SP

```
df_top_seller = df_master.groupby('seller_id')['order_item_total'].sum(). \
    reset_index(name='revenue'). \
    sort_values(by='revenue', ascending= False)
df_top_seller.head()
```

	seller_id	revenue
857	4869f7a5dfa277a7dca6462dcf3b52b2	249640.70
1535	7c67e1448b00f6e969d365cea6b010ab	239536.44
1013	53243585a1d6dc2643021fd1853d8905	235856.68
881	4a3ca9315b744ce9f8e9374361493884	235539.96
3024	fa1c13f2614d7b5c4749cbc52fecda94	204084.73

✓ • Seller contribution to revenue

```
df_top_seller['contribution_pct'] = df_top_seller['revenue']*100/df_top_seller['revenue'].sum()
```

```
df_top_seller.head()
```

	seller_id	revenue	contribution_pct
857	4869f7a5dfa277a7dca6462dcf3b52b2	249640.70	1.575661
1535	7c67e1448b00f6e969d365cea6b010ab	239536.44	1.511886
1013	53243585a1d6dc2643021fd1853d8905	235856.68	1.488660
881	4a3ca9315b744ce9f8e9374361493884	235539.96	1.486661
3024	fa1c13f2614d7b5c4749cbc52fecda94	204084.73	1.288125

✓ • Seller distribution

```
# which state has most sellers
```

```
df_master.groupby('seller_state')['order_item_total'].count().reset_index(name = 'num_of_seller')
.sort_values(by = 'num_of_seller', ascending = False) \
.head(10)
```

	seller_state	num_of_seller
22	SP	80342
8	MG	8827
15	PR	8671
16	RJ	4818
20	SC	4075
19	RS	2199
4	DF	899
2	BA	643
6	GO	520
13	PE	448

Review and Satisfaction Analysis

- Distribution of review scores
- Relationship between delivery time and ratings
- Identification of dissatisfaction patterns

✓ Distribution of review scores

```
df_review_score = df_master['review_score'][df_master['review_score'].notna()].astype('int').value_counts()
df_review_score['review_score_percent'] = (df_review_score['count'] * 100/df_review_score['count'].sum())
df_review_score.head()
```

	review_score	count	review_score_percent
0	5	63152	56.15
1	4	21267	18.91
2	1	14635	13.01
3	3	9469	8.42
4	2	3941	3.50

✓ Relationship between delivery time and ratings

```
df_delivery_rating = df_master[['delivery_time', 'review_score']].dropna()
```

```
df_delivery_rating.groupby('review_score')['delivery_time'].mean()
```

```
review_score
1.000000    16.149826
1.500000    14.388889
2.000000    14.367394
2.500000    14.152174
3.000000    13.199852
3.333333    22.000000
3.500000    11.612903
4.000000    11.652845
4.333333     3.000000
4.500000    12.264706
```

```
5.000000    10.131381
Name: delivery_time, dtype: float64
```

```
df_delivery_rating[['delivery_time', 'review_score']].corr()
```

	delivery_time	review_score
delivery_time	1.000000	-0.220849
review_score	-0.220849	1.000000

```
df_delivery_rating['delivery_bucket'] = pd.cut(
    df_delivery_rating['delivery_time'],
    bins=[0, 3, 7, 14, 30, 100],
    labels=['0-3 days', '3-7 days', '7-14 days', '14-30 days', '30+ days']
)
```

```
df_delivery_rating.groupby('delivery_bucket')['review_score'].mean()
```

C:\Users\atanu\AppData\Local\Temp\ipykernel_25940\2255627614.py:1: FutureWarning: The default of c
df_delivery_rating.groupby('delivery_bucket')['review_score'].mean()
delivery_bucket
0-3 days 4.393801
3-7 days 4.309693
7-14 days 4.199082
14-30 days 3.843342
30+ days 2.154975
Name: review_score, dtype: float64

✓ • Identification of dissatisfaction patterns

```
df_dissat = df_master.copy()
```

```
df_dissat['is_dissatisfied'] = df_dissat['review_score'] <= 2
```

```
# delivery time to is_dissatisfied
df_dissat.groupby('is_dissatisfied')['review_score'].mean()
```

```
is_dissatisfied
False    4.571297
True     1.210685
Name: review_score, dtype: float64
```

```
df_dissat['delay'] = ( df_dissat['order_delivered_customer_date'] - df_dissat['order_purchase_time'] )
df_dissat.groupby('review_score')['delay'].mean()
```

```
review_score
1.000000    19.157527
1.500000    14.388889
2.000000    15.354098
2.500000    16.000000
3.000000    13.564069
3.333333    22.000000
3.500000    12.482759
4.000000    11.747861
4.333333     3.000000
4.500000    12.462687
5.000000    10.180421
Name: delay, dtype: float64
```

```
df_dissat[df_dissat['is_dissatisfied']]['product_category_name_english'].value_counts().head(10)
```

```

product_category_name_english
bed_bath_table           2058
furniture_decor          1587
computers_accessories    1439
health_beauty            1312
sports_leisure           1252
housewares               1079
watches_gifts            965
telephony                761
garden_tools             689
auto                     637
Name: count, dtype: int64

```

✓ Step 6: Data Visualization

Use Matplotlib and Seaborn to create:

- Time series plots (sales trends)
- Bar charts (category performance)
- Histograms (distribution analysis)
- Box plots (outlier detection)
- Heatmaps (correlation analysis)

All visualizations must be clearly labeled and interpretable.

```

import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

```

✓ Time series plots(sales trends)

```

# We are trying to find out how sales figure varies across months
df_master['order_purchase_timestamp'] = pd.to_datetime(df_master['order_purchase_timestamp'])
df_master['order_month'] = df_master['order_purchase_timestamp'].dt.to_period('M')

```

```

df_sales_trend = df_master.groupby('order_month')['order_item_total'].sum().reset_index()
df_sales_trend.head()

```

	order_month	order_item_total
0	2016-09	354.75
1	2016-10	56808.84
2	2016-12	19.62
3	2017-01	137188.49
4	2017-02	286280.62

```
plt.figure()
df_sales_trend['order_month'] = df_sales_trend['order_month'].dt.to_timestamp()
sns.lineplot(data=df_sales_trend, x='order_month', y='order_item_total', marker='o')

plt.title("Sales Trend")
plt.xlabel('Order Month')
plt.ylabel('Revenue')
plt.xticks(rotation=45)
plt.tight_layout()
```



- Business grows very fast around first 3 quarters of 2017, reaching peak during festive seasons.
- In 2018 the sales is stable.
- Then suddenly it drops to minimum, that means there may be data issue like missing data.

✓ Bar Chart (Category Performance)

```
df_top_category = df_master.groupby('product_category_name_english')['order_item_total'] \
    .sum().sort_values(ascending=False).head(10).reset_index()
```

```
plt.figure()
sns.barplot(data=df_top_category, x='order_item_total', y='product_category_name_english')
plt.title('Top Ten Categories')
plt.xlabel('Revenue')
plt.ylabel('Category')
plt.show()
```